UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional San Nicolás

Tecnicatura Universitaria en Programación



TurnoFácil (TF)

Sistema de Gestión de Horarios y Reservas

Gestión de Proyectos - Proyecto Final

Docente Teoría: Ing. David Cecchi

Docente Práctica: A.U.S. Juan Arce

Alumno: Dante Orsetti

2025

1. Nombre

Turno Fácil: Sistema para gestión de clases, reservas y pagos en negocios con actividades de cupos limitados.

2. Siglas

TurnoFácil

3. Descripción

TurnoFácil es un sistema web responsivo destinado a gestionar horarios, actividades, clases, alumnos, profesores, reservas y pagos en negocios que ofrecen actividades con cupos limitados. Su caso inicial corresponde a un local de pilates ubicado en Empalme Villa Constitución. La solución fue diseñada para poder adaptarse a otros negocios de características similares.

La aplicación utiliza una arquitectura multi-tenant: cada negocio se administra como un tenant y sus datos deben mantenerse separados de los de los demás negocios. Los usuarios autorizados pueden administrar las actividades y clases del negocio. Los alumnos pueden consultar las clases disponibles y reservarlas cuando cumplen las validaciones del sistema, que incluyen pertenencia al negocio, cupos, ausencia de una reserva duplicada y condiciones del plan y del ciclo de 30 días.

El sistema incluye además funcionalidades de pagos, listas de espera, novedades, invitaciones, ejercicios, rutinas y permisos y estadísticas para el dueño. El desarrollo se realizó como proyecto académico de la Tecnicatura Universitaria en Programación.

4. Objetivos

- Administrar las actividades, especialidades, profesores y clases de cada negocio desde una aplicación centralizada.
- Facilitar que los alumnos consulten clases disponibles y registren reservas de acuerdo con las condiciones de su plan y ciclo mensual.
- Controlar la capacidad máxima de las clases y evitar reservas duplicadas para un mismo alumno y una misma clase.
- Permitir cancelar reservas de acuerdo con el límite temporal implementado.
- Gestionar y consultar los pagos de alumnos al negocio, así como los pagos del negocio a TurnoFácil.
- Proporcionar una solución multi-tenant adaptable a distintos negocios, manteniendo separados sus datos y el acceso a sus funcionalidades.
- Facilitar la comunicación operativa mediante correos de verificación de cuenta, recuperación de contraseña y avisos de disponibilidad para alumnos en lista de espera.
- Proporcionar al dueño información estadística sobre alumnos, reservas e ingresos registrados para apoyar la toma de decisiones.

5. Definición de Requerimientos

5.1. Requerimientos Funcionales

- Registrar usuarios, verificar el correo e iniciar sesión mediante los mecanismos de autenticación implementados, incluida la autenticación con Google.
- Permitir las operaciones disponibles sobre la cuenta del usuario, como modificar datos de perfil, cambiar contraseña y solicitar recuperación de contraseña.
- Crear y gestionar negocios como tenants, incluyendo la configuración y pertenencia implementadas.
- Gestionar alumnos de un negocio y asignarles un plan de alumnos.

- Gestionar profesores de un negocio, sus especialidades y su estado activo o inactivo.
- Gestionar las especialidades de un negocio.
- Gestionar las actividades de un negocio.
- Crear, consultar, modificar y eliminar clases con fecha, horario, capacidad y actividad, y administrar clases modelo según las operaciones disponibles.
- Registrar reservas. Validar pertenencia al tenant, clase y alumno existentes, que la clase no haya pasado ni comenzado, ausencia de duplicados, cupo disponible, plan válido, cuota al día, ciclo activo y límite de clases del ciclo.
- Consultar reservas por alumno, clase o identificador según las operaciones expuestas por la API.
- Eliminar una reserva propia. No se permite cancelar durante los 20 minutos previos al inicio de la clase.
- Permitir incorporarse a la lista de espera, cuando corresponda, y evitar entradas duplicadas para la misma clase.
- Registrar y consultar pagos de alumnos al negocio, con los datos disponibles de importe, fecha, medio, plan y estado.
- Gestionar el registro de los pagos del negocio por su plan de TurnoFácil. El registro manual por administración está contemplado. El cobro automático mediante Mercado Pago esté implementado de extremo a extremo.
- Gestionar grupos, asignaciones de usuarios a grupos y permisos de grupos, además de permisos individuales de profesores según las funciones implementadas.
- Publicar y consultar novedades y registrar lecturas según las operaciones disponibles.
- Gestionar ejercicios y rutinas, con acceso sujeto a las condiciones del plan del negocio implementadas.
- Crear invitaciones e incorporar usuarios a un negocio mediante el flujo de invitación implementado.
- Permitir al dueño consultar estadísticas disponibles para su negocio: resumen de alumnos activos, reservas e ingresos registrados, altas y evolución mensual de alumnos, alumnos por plan y datos de profesores.

5.1. Requerimientos No Funcionales

- La aplicación debe ofrecer una interfaz web responsiva, utilizable desde navegadores de escritorio y dispositivos móviles.
- El acceso a funciones y datos debe estar protegido mediante autenticación y autorización, respetando los permisos definidos.
- Las operaciones asociadas a un negocio deben verificar el contexto y la pertenencia al tenant correspondiente.
- El sistema debe permitir múltiples accesos simultáneos sin degradar su funcionamiento.
- Las notificaciones y confirmaciones deben enviarse en tiempo real o con un retraso máximo de 1 minuto.
- Las reservas, los pagos y los datos del negocio deben persistirse para su consulta posterior.

Minutas de Reunión

MR-01

- Fecha: 24/09/2025
- Hora: 16:00 – 17:00
- Lugar: Local de Pilates, Empalme Villa Constitución
- Participantes:
  - Laura Martínez (Administradora del local)
  - Dante Orsetti (Desarrollador del sistema)

Temas tratados:

- Definición de la funcionalidad de inicio de sesión para administradores y alumnos.
- Revisión de la gestión de planes de alumnos (ejemplo: 2 veces por semana, ilimitado, etc.).
- Necesidad de controlar que los alumnos solo puedan reservar la cantidad de clases que permite su plan.
- Discusión sobre la opción de que el administrador pueda modificar manualmente un plan en casos excepcionales. Acuerdos / Decisiones.
- El sistema contará con un módulo de inicio de sesión con credenciales únicas para cada usuario.

- Se implementará la gestión de planes: cada alumno tendrá asociado un tipo de plan que definirá cuántas clases puede tomar por semana.
- El sistema bloqueará automáticamente reservas que excedan el límite del plan contratado.
- El administrador tendrá permisos especiales para modificar el plan de un alumno cuando sea necesario.

Tareas asignadas

- Dante: Diseñar el prototipo de la pantalla de inicio de sesión y de la gestión de planes.
- Laura: Definir los distintos tipos de planes que ofrecerá el local (ejemplo: 2, 3 veces por semana).

MR-02

- Fecha: 10/09/2025
- Hora: 16:00 – 17:00
- Lugar: Local de Pilates, Empalme Villa Constitución
- Participantes:
  - Laura Martínez (Administradora del local)
  - Dante Orsetti (Desarrollador del sistema)

Temas tratados

- Necesidad de gestionar horarios y turnos de clases.
- Requisitos para que los alumnos puedan reservar, cancelar o reprogramar turnos.
- Integración de pagos dentro de la aplicación.
- Control de cantidad máxima de alumnos por clase.
- Acuerdos / Decisiones El sistema permitirá que los alumnos solo puedan reservar o modificar turnos si tienen la cuota mensual abonada.
- Se implementará un módulo de pagos seguro dentro de la app.
- El administrador podrá cargar el pago de cualquier alumno si el pago es de diferente manera

Tareas asignadas

- Dante: Crear prototipos de interfaz para reservas, pagos y gestión de clases.
- Laura: Proveer listado de actividades y horarios actuales para el sistema.

6.Alcance:

Inclusiones:

- Autenticación y gestión de usuarios, alumnos, profesores y negocios.
- Administración de actividades, especialidades, clases y clases modelo.
- Gestión de planes de alumnos con límites definidos mediante ClassesPerMonth.
- Consulta de clases y registro de reservas con validación de tenant, cupo, duplicidad, plan y ciclo mensual de 30 días.
- Cancelación de reservas antes del límite temporal implementado.
- Listas de espera para clases completas y notificación por correo cuando se libera un lugar; el sistema no asigna automáticamente una reserva.
- Registro y consulta de pagos, incluyendo el flujo de pago de alumnos al negocio y la carga manual disponible.
- Novedades, invitaciones, ejercicios y rutinas según los permisos y condiciones del plan del negocio.
- Gestión de grupos y permisos, incluidos permisos individuales de profesores.
- Estadísticas disponibles para el dueño: alumnos, reservas, ingresos registrados, evolución de altas y distribución por plan.
- Aislamiento de datos por tenant mediante contexto, middleware y validaciones en los servicios.

Exclusiones:

- Reprogramación de reservas: no implementada.
- Validación de superposición horaria entre reservas del mismo alumno: no implementada.
- Registro de asistencia efectiva a clases y cálculo de ocupación real: no implementados.
- Gestión de varias sedes físicas dentro de un mismo tenant.
- Integración con sistemas contables externos.
- Integración con redes sociales para difusión o recordatorios.

7. Registro de Interesados

Nombre	Organización / localización	Rol	Expectativas principales	Influencia / fase de interés	Tipo / posición
Laura Martínez	Local de pilates, Empalme Villa Constitución	Administradora / usuaria referente	Organizar clases, planes, reservas y pagos.	Influencia alta; relevamiento, validación funcional.	Externa al equipo de desarrollo / apoyo.
Dante Orsetti	UTN Facultad Regional San Nicolás; desarrollo del proyecto	Desarrollador y responsable del proyecto	Construir, probar y documentar el sistema.	Influencia alta; todas las fases.	Interno al proyecto / apoyo.
Alumnos de los negocios	Usuarios finales; ubicación según cada negocio	Reservar clases y consultar pagos y reservas.	Consultar disponibilidad y reservar de manera sencilla.	Influencia indirecta; diseño, pruebas y uso.	Externos.
Profesores de los negocios	Usuarios de cada tenant	Gestionar las funciones habilitadas para profesores.	Acceder a las herramientas necesarias según sus permisos.	Influencia media; diseño y operación.	Externos.
Dueños de otros negocios	Negocios potenciales que utilicen la plataforma	Interesados potenciales	Gestionar su negocio en un entorno separado.	Influencia potencial; evolución futura.	Externos.
Administración de TurnoFácil	Organización administradora de la plataforma; datos formales por completar	Administración de plataforma	Gestionar negocios y suscripciones según las funciones implementadas.	Influencia alta en operación de plataforma.	Interno.
Juan Arce	UTN Facultad Regional San Nicolás	Tutor y guía academico	Que el proyecto cumpla con los objetivos academicos, profesionales y técnicos.		Interno / apoyo.

8. Cronograma de Hitos

Los rangos siguientes corresponden a registros de cambios asociados a módulos del repositorio y pueden solaparse. Las fechas de relevamiento y prototipos son estimaciones retrospectivas.

Código	Hito	Fecha / rango	Entregable
H-01	Relevamiento y definición inicial	Octubre de 2025	Requerimientos iniciales y minutas.
H-02	Prototipos iniciales	Noviembre de 2025 (estimado)	Prototipos de pantallas principales.
H-03	Usuarios, autenticación, negocios y planes	Cierre del núcleo: 03/04/2026	Persistencia inicial, acceso a los modulos
H-04	Profesores, especialidades, actividades y clases	Cierre del núcleo: 10/03/2026	Gestión inicial de clases y módulos relacionados.
H-05	Reservas	Cierre del núcleo: 11/03/2026	Registro de reservas y validaciones; mejoras posteriores.
H-06	Pagos	Cierre del núcleo: 04/03/2026	Servicios de pago y mejoras posteriores.
H-07	Funcionalidades complementarias	Rango aproximado: 16/07/26–22/10/26	Invitaciones, listas de espera, novedades, ejercicios, rutinas, estadísticas y permisos.
H-08	Cierre	Entrega informada: 23/10/2026	Versión final indicada para entrega.

9. Criterios de Aceptación

- Las operaciones protegidas deben requerir autenticación y respetar los permisos del usuario.
- Las operaciones asociadas a un negocio deben validar el tenant correspondiente y no permitir que el encabezado X-Tenant-Id, por sí solo, otorgue acceso.
- El sistema debe permitir consultar clases y calcular disponibilidad a partir de MaxCapacity y las reservas existentes.

- Al registrar una reserva, el sistema debe validar pertenencia al tenant, clase y alumno existentes, clase futura, ausencia de reserva duplicada, cupo, plan válido, cuota al día, ciclo activo y límite del ciclo de 30 días.
- Una clase completa debe rechazarse como reserva directa y la interfaz debe ofrecer el flujo de lista de espera cuando corresponda.
- La cancelación debe respetar la regla temporal: no se permite cancelar durante los 20 minutos previos al inicio.
- Las reservas canceladas mediante el flujo actual deben eliminarse; no se debe representar una cancelación como estado persistente si el registro se elimina.
- El sistema debe permitir registrar y consultar pagos mediante las operaciones disponibles.
- El dueño debe poder consultar las estadísticas implementadas correspondientes a su negocio.
- Los datos de un negocio no deben quedar accesibles desde el contexto de otro tenant en los flujos protegidos.

10.Supuestos del proyecto:

- Los usuarios cuentan con acceso a Internet y un navegador compatible.
- Los dueños cargan correctamente las actividades, clases, planes y datos de sus negocios.
- Los proveedores externos utilizados, como el proveedor de correo y Mercado Pago, se encuentran disponibles y las credenciales están configuradas.
- Cada negocio opera como un tenant independiente.
- Los datos ingresados sobre pagos, planes y horarios son correctos y se mantienen actualizados por los usuarios autorizados.

11.Restricciones del proyecto

- Tiempo: el desarrollo se realiza dentro del período académico destinado al Proyecto Final.
- Recursos humanos: el proyecto cuenta con un único desarrollador principal.
- Presupuesto: se priorizan herramientas y servicios accesibles.
- Tecnología: el sistema depende de las tecnologías y proveedores integrados en la versión actual.
- Alcance: las funcionalidades deben ajustarse a lo implementado y no se consideran incluidas la reprogramación ni la validación de superposición horaria.

12. Iteraciones

El desarrollo se organizó en iteraciones funcionales. Los módulos tuvieron ampliaciones posteriores.

Iteración 1 – Usuarios y negocios

- Registro y autenticación.
- Gestión de usuarios y negocios (tenants).
- Configuración inicial de planes y pertenencia.

Iteración 2 – Actividades y clases

- Gestión de profesores y especialidades.
- Gestión de actividades y clases.
- Clases modelo y ampliaciones de programación.

Iteración 3 – Reservas de clase (core)

- Consulta de clases disponibles.
- Registro de reservas con validaciones de negocio.
- Consulta y cancelación de reservas.
- Listas de espera para clases completas.

Iteración 4 – Pagos

- Registro y consulta de pagos.
- Actualización del estado de cuota.
- Integración con Mercado Pago en los flujos implementados.

Iteraciones complementarias

- Novedades, invitaciones, ejercicios y rutinas.
- Permisos de acceso y permisos individuales de profesores.
- Estadísticas del negocio.

12.1. Especificación del requerimiento core ID: Registrar reservas de clases.

Objetivo: permitir que un alumno autenticado registre una reserva de una clase futura de su negocio, siempre que cumpla las validaciones de acceso, disponibilidad y plan.

Actor principal: alumno autenticado.

Precondiciones: el usuario está autenticado; el alumno pertenece al tenant solicitado; la clase existe y pertenece al mismo tenant; la clase no ha pasado ni comenzado; el alumno cuenta con un plan/ciclo válido y la cuota no está vencida.

Disparador: el alumno selecciona una clase disponible y pulsa la acción para unirse.

Flujo principal:

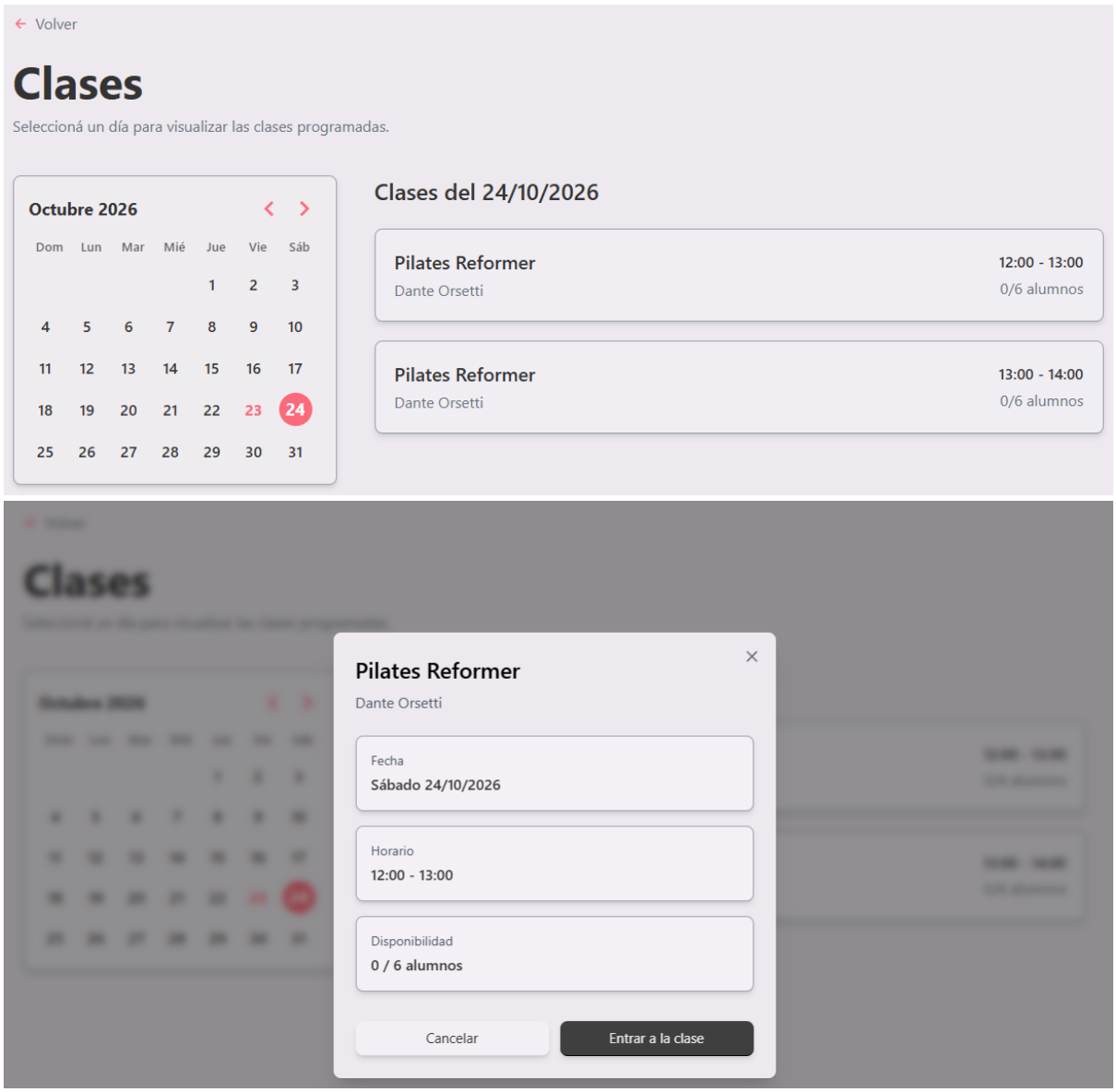
El alumno abre la sección de clases de su negocio. El sistema presenta clases y disponibilidad calculada a partir de la capacidad máxima y las reservas existentes. El alumno selecciona una clase y solicita reservarla. El frontend envía POST /api/reservations/bulk con un BulkCreateReservationDTO. En una reserva individual, StudentIds contiene un único alumno. El backend valida permisos, tenant, clase, alumno, fecha, cupo, duplicidad, plan, estado de cuota y ciclo de 30 días. Si las validaciones son satisfactorias, se crea la reserva con estado Pending. El frontend informa que el alumno se unió correctamente y cierra el modal de confirmación después de aproximadamente dos segundos. Postcondición: la reserva queda persistida y la disponibilidad se recalcula a partir de las reservas existentes. No se actualiza un contador persistente de cupos.

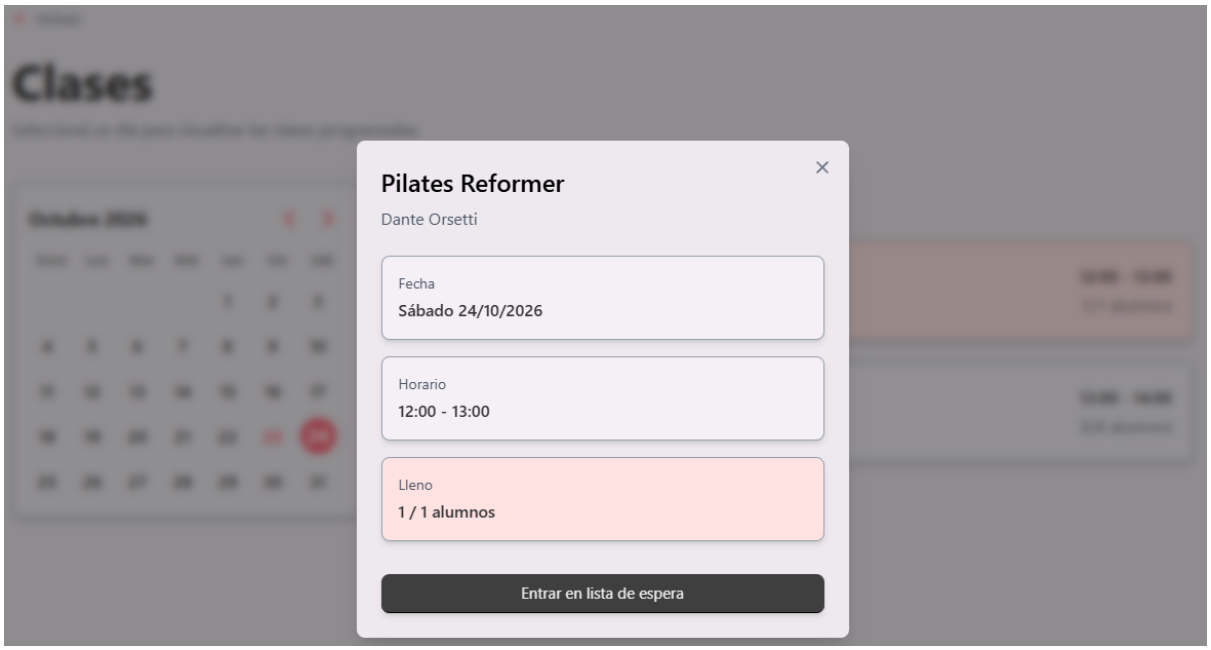
Reglas de negocio:

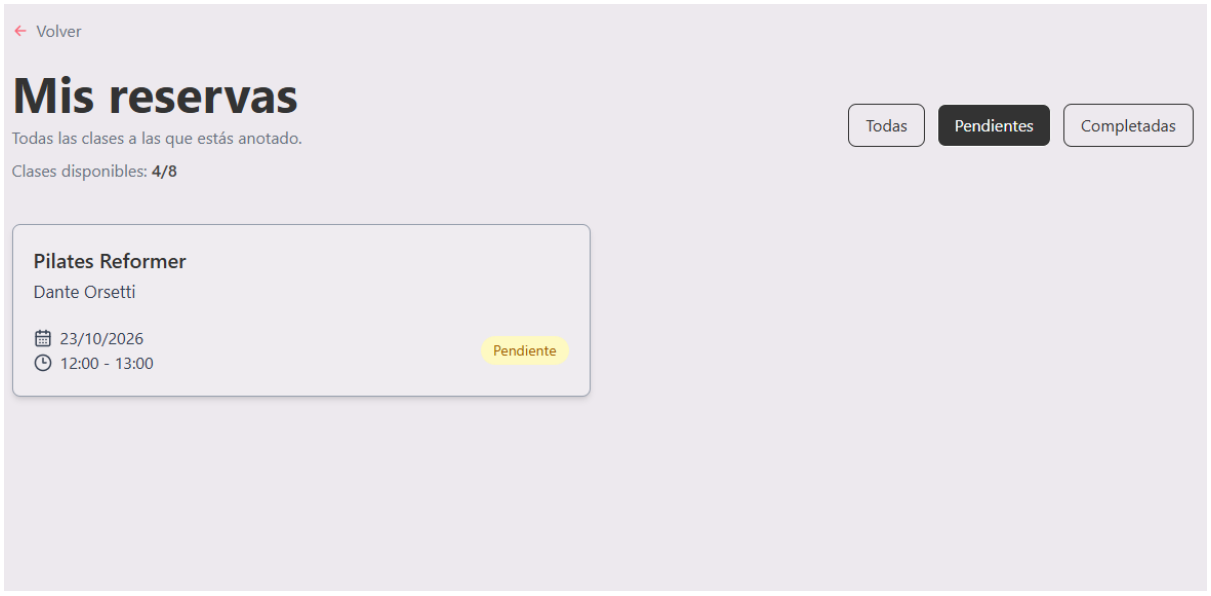
No se permite reservar una clase pasada o que ya comenzó. No se permite más de una reserva del mismo alumno para la misma clase. No se permite superar MaxCapacity. El alumno debe pertenecer al tenant de la clase. El alumno debe tener un plan y ciclo válidos, cuota al día y no haber alcanzado ClassesPerMonth dentro del ciclo de 30 días vinculado a PaymentDueDate. Una clase completa utiliza el flujo separado de lista de espera. La cancelación elimina la reserva y no está permitida durante los 20 minutos previos al inicio.

12.2. Guion de la interfaz de usuario

El alumno ingresa al negocio correspondiente y abre la sección Clases. Consulta las clases disponibles. Las tarjetas muestran la actividad, profesor, fecha, horario y relación entre reservas y capacidad (por ejemplo, 3/10). Selecciona una clase para consultar el detalle. Pulsa «Entrar a la clase». Si la reserva es válida, el sistema muestra «¡Te uniste a la clase correctamente!» y cierra el modal aproximadamente a los dos segundos. Si la clase está completa, la interfaz ofrece «Entrar en lista de espera» cuando corresponde. El alumno puede consultar sus reservas en «Mis reservas», con filtros disponibles y un mensaje de estado vacío cuando no hay reservas. Si ocurre un error, se muestra el mensaje específico del backend cuando está disponible; en caso contrario, se utiliza un mensaje genérico.


Volver
Clases
Seleccioná un día para visualizar las clases programadas.
Octubre 2026
Dom Lun Mar Mié Jue Vie Sáb
1 2 3
4 5 6 7 8 9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
Clases del 24/10/2026
Pilates Reformer
Dante Orsetti
12:00 - 13:00
0/6 alumnos
Pilates Reformer
Dante Orsetti
13:00 - 14:00
0/6 alumnos
Pilates Reformer
Dante Orsetti
Fecha
Sábado 24/10/2026
Horario
12:00 - 13:00
Disponibilidad
0 / 6 alumnos
Cancelar
Entrar a la clase

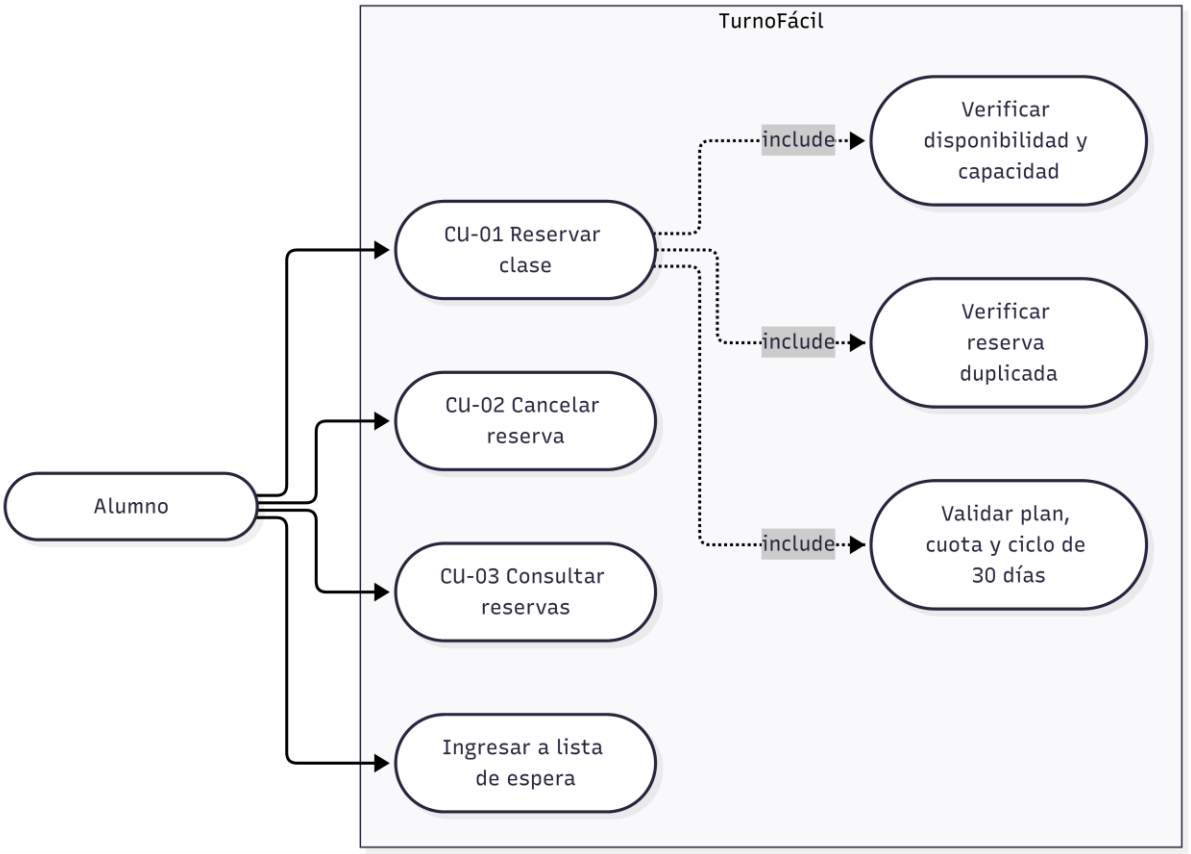

Clases
Pilates Reformer
Dante Orsetti
Fecha
Sábado 24/10/2026
Horario
12:00 - 13:00
Lleno
1 / 1 alumnos
Entrar en lista de espera


Volver
Mis reservas
Todas las clases a las que estás anotado.
Clases disponibles: 4/8
Todas
Pendientes
Completadas
Pilates Reformer
Dante Orsetti
23/10/2026
12:00 - 13:00
Pendiente

12.3. Análisis de requisitos

12.3.1. Modelo de casos de uso

12.3.1.1. Diagramas de caso de uso



3.1.2. Especificación de caso de uso

CU-01 — Registrar reserva

Actor principal: alumno autenticado. Referencia: RF-09. Precondiciones: alumno y clase válidos, pertenencia al tenant, plan/ciclo vigente y cuota al día.

Flujo principal: el alumno selecciona una clase; el sistema muestra detalle y disponibilidad; el alumno confirma; el frontend envía POST /api/reservations/bulk; el backend valida las reglas; persiste la reserva Pending; la interfaz muestra confirmación.

Flujos alternativos: si no hay cupo, se ofrece lista de espera; si la clase pasó o comenzó, se rechaza; si existe reserva duplicada, se rechaza; si el plan, cuota o ciclo no es válido, se informa el motivo; si el alumno no pertenece al tenant, se deniega; si falla la persistencia, se muestra un error.

CU-02 — Cancelar reserva

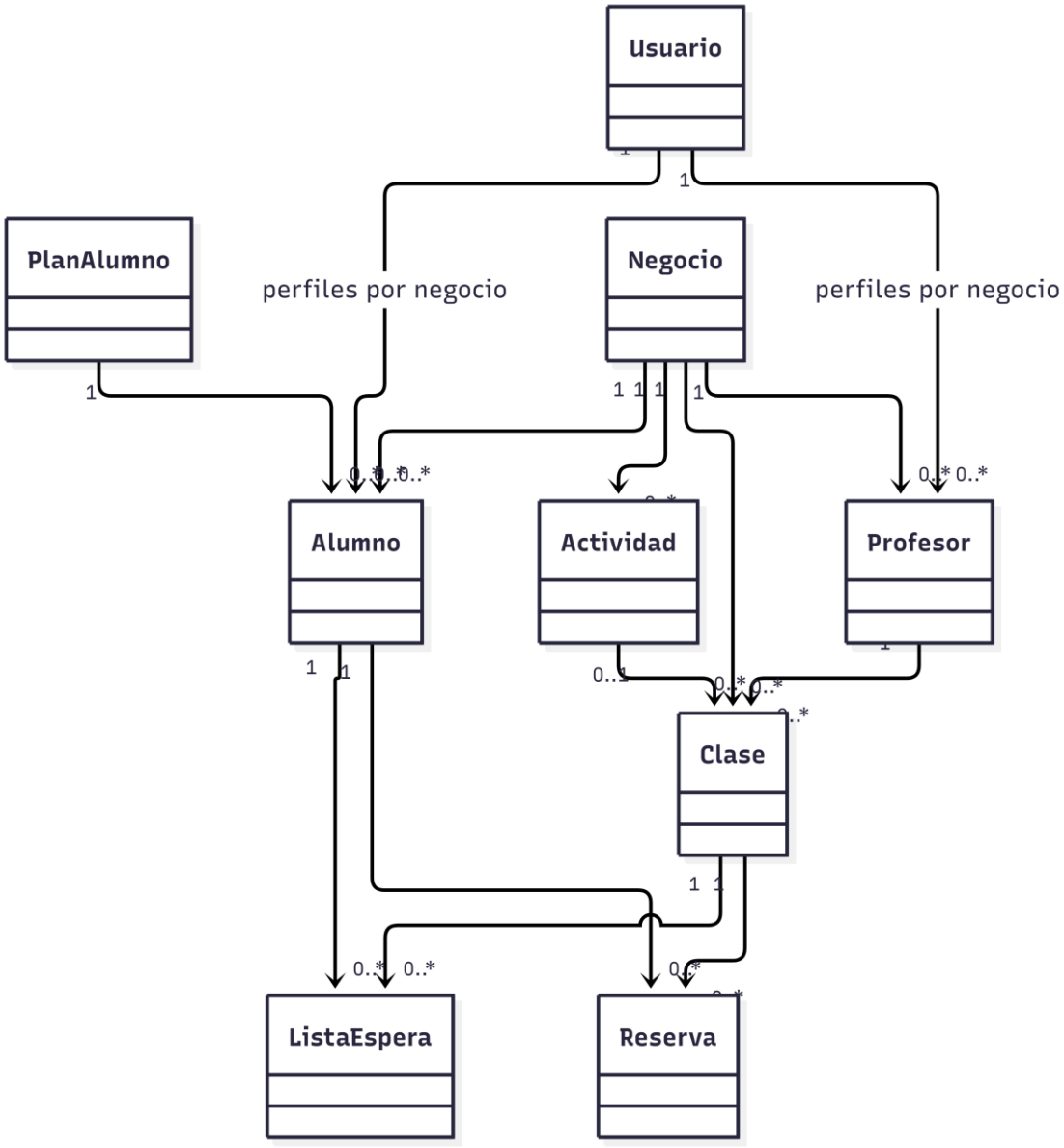
Actor principal: alumno autenticado. El alumno selecciona una reserva propia y solicita cancelarla. El backend verifica acceso y límite temporal. Si se permite, elimina el registro. No se guarda un estado Cancelada. Si la eliminación libera lugar en una clase con lista de espera, se activa el mecanismo de notificación por correo.

CU-03 — Consultar reservas

Actor principal: alumno autenticado. El alumno accede a «Mis reservas». El sistema recupera las reservas asociadas al alumno y las presenta con los filtros y orden disponibles. La disponibilidad de clases se calcula con la capacidad máxima y las reservas existentes.

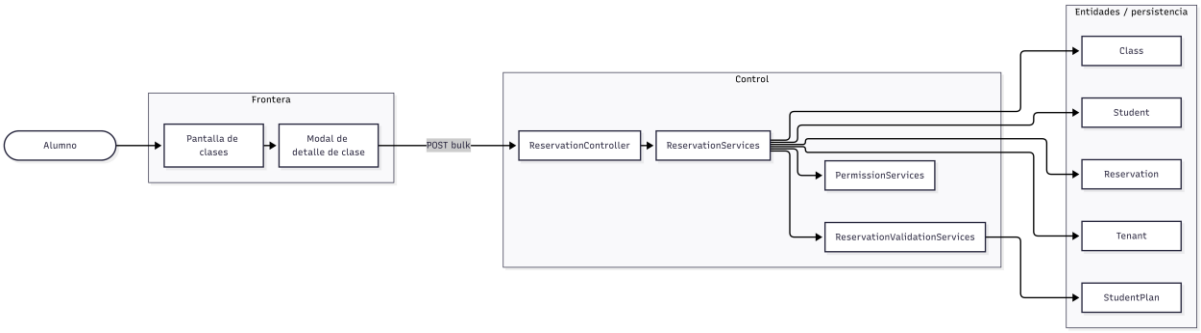
3.2. Modelo de dominio

3.2.1. Diagrama de dominio conceptual del problema

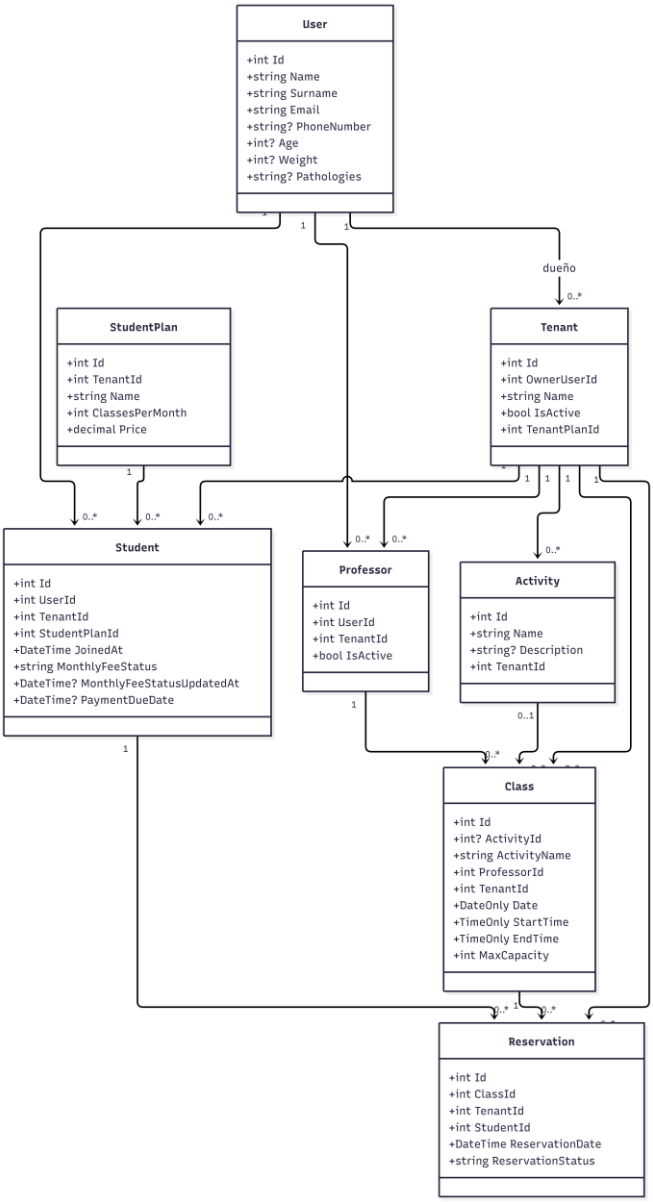


4. Análisis del diseño preliminar

4.1. Diagramas de robustez



4.2. Diagrama de dominio actualizado

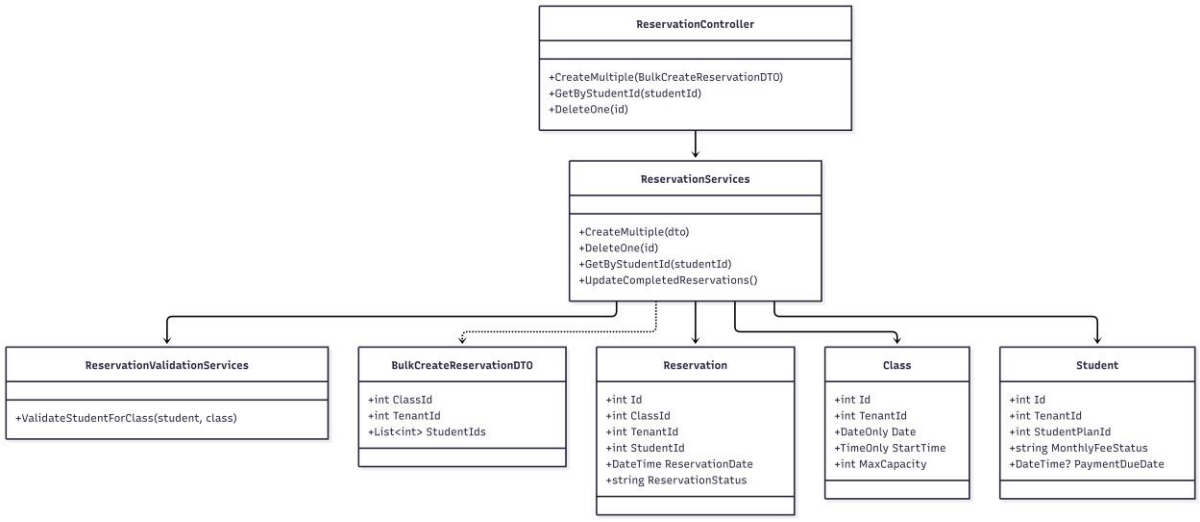


5. Diseño detallado

5.1. Diagramas de secuencia



5.2. Diagrama de clases del flujo



6. Implementación

6.1. Código

Endpoint del Contoller de las reservas:

```
[HttpPost("bulk")]
[Authorize]
[ProducesResponseType(typeof(List<ResponseReservationDTO>), StatusCodes.Status200OK)]
[ProducesResponseType(typeof(HttpMessage), StatusCodes.Status400BadRequest)]
[ProducesResponseType(typeof(HttpMessage), StatusCodes.Status404NotFound)]
[ProducesResponseType(typeof(HttpMessage), StatusCodes.Status500InternalServerError)]
0 referencias
public async Task<ActionResult<List<ResponseReservationDTO>>> CreateMultiple([FromBody] BulkCreateReservationDTO bulkDTO)
{
    try
    {
        var reservations = await _reservationServices.CreateMultiple(bulkDTO);
        return Ok(reservations);
    }
    catch (HttpResponseError ex)
    {
        return StatusCode((int)ex.StatusCode, new { message = ex.Message });
    }
    catch (Exception ex)
    {
        return StatusCode((int)HttpStatusCode.InternalServerError, new { message = ex.Message });
    }
}
```

Método para crear un registro resumido (sin verificaciones de errores):

```
public async Task<List<ResponseReservationDTO>> CreateMultiple(
    BulkCreateReservationDTO bulkDTO)
{
    await _permissionServices.CheckPermission(
        Permissions.RESERVATION_CREATE
    );
    var tenantId = _currentTenantService.GetRequiredTenantId();
    var classEntity = await _classRepository.GetOneAsync(c => c.Id == bulkDTO.ClassId, c => c.Activity);

    var reservations = new List<Reservation>();
    var errors = new List<string>();

    foreach (var studentId in bulkDTO.StudentIds)
    {
        var student =await _studentRepository.GetOneAsync(s => s.Id == studentId,s => s.User);
        var existingReservation = await _reservationRepository.GetOneAsync(r =>r.ClassId == bulkDTO.ClassId && r.StudentId == studentId);
        var reservationsCount =await _reservationRepository.CountAsync(r => r.ClassId == bulkDTO.ClassId);
        var validationError =await _reservationValidationServices.ValidateStudentForClass(student,classEntity);

        reservations.Add(
            new Reservation
            {
                ClassId = bulkDTO.ClassId,
                TenantId = tenantId,
                StudentId = studentId,
                ReservationDate = DateTime.UtcNow,
                ReservationStatus =
                ReservationStatus.PENDING
            }
        );
    }
    foreach (var reservation in reservations)
    {
        await _reservationRepository.CreateOneAsync(reservation);
    }

    return _mapper.Map<List<ResponseReservationDTO>>(reservations);
}
```

Modelos de reserva y clases:

```
public class Reservation
{

    public int Id { get; set; }

    public int ClassId { get; set; }

    public Class Class { get; set; } = null!;

    public int TenantId { get; set; }

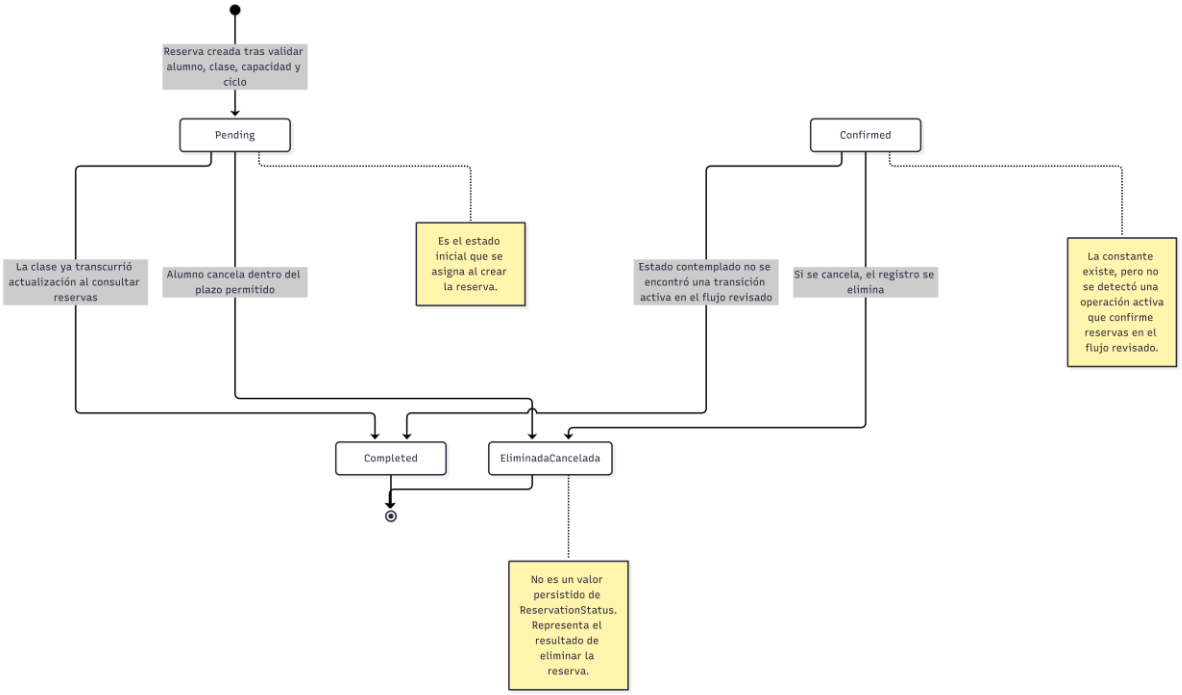
    public Tenant Tenant { get; set; } = null!;
    10 referencias
    public int StudentId { get; set; }
    10 referencias
    public Student Student { get; set; } = null!;
    3 referencias
    public DateTime ReservationDate { get; set; }
    12 referencias
    public string ReservationStatus { get; set; } = null!;
}
```

```
public class Class
{
    25 referencias
    public int Id { get; set; }
    8 referencias
    public int? ActivityId { get; set; }
    11 referencias
    public Activity? Activity { get; set; }
    6 referencias
    public string ActivityName { get; set; } = null!;
    5 referencias
    public int ProfessorId { get; set; }
    9 referencias
    public Professor Professor { get; set; } = null!;
    16 referencias
    public int TenantId { get; set; }
    4 referencias
    public Tenant Tenant { get; set; } = null!;
    35 referencias
    public DateOnly Date { get; set; }
    18 referencias
    public TimeOnly StartTime { get; set; }
    8 referencias
    public TimeOnly EndTime { get; set; }
    10 referencias
    public int MaxCapacity { get; set; }
    19 referencias
    public ICollection<Reservation> Reservations { get; set; } = new List<Reservation>();
    2 referencias
    public ICollection<Waitlist> Waitlists { get; set; } = new List<Waitlist>();
}
```

6.2. Persistencia



7.Diagrama de Transición de Estados

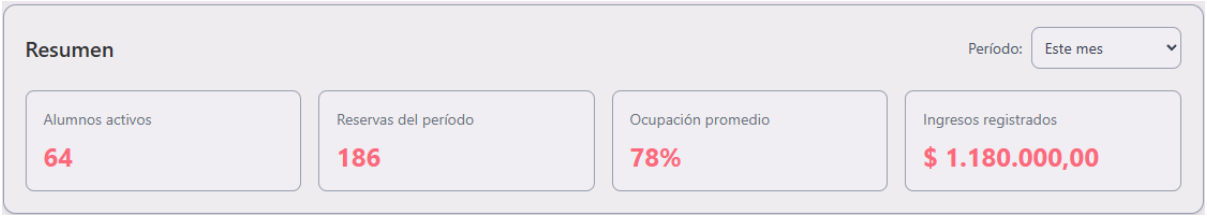


13. Reportes, indicadores y estadísticas

El sistema incluye un módulo de estadísticas para el dueño del negocio. La API utiliza StatisticsController.cs y StatisticsService.cs, y el frontend se encuentra en statistics.jsx. Las consultas están asociadas al tenant y la autorización verifica que el usuario tenga acceso como dueño.

13.1. Resumen

Cantidad de alumnos activos. Cantidad de reservas. Ocupación promedio. Ingresos registrados por pagos de alumnos al negocio. Filtros temporales disponibles: este mes, últimos tres meses, este año y todo el período.



13.2. Estadísticas de clases

Ocupación por día y por actividad. Clases programadas. Reservas realizadas. Capacidad promedio según el período seleccionado.



13.3. Estadísticas de alumnos

Cantidad de alumnos activos. Altas de alumnos durante este mes, los últimos tres meses y este año. Evolución mensual de nuevas altas durante los últimos doce meses. Distribución de alumnos por plan. La evolución mensual representa nuevas incorporaciones por mes, no el total acumulado de alumnos.



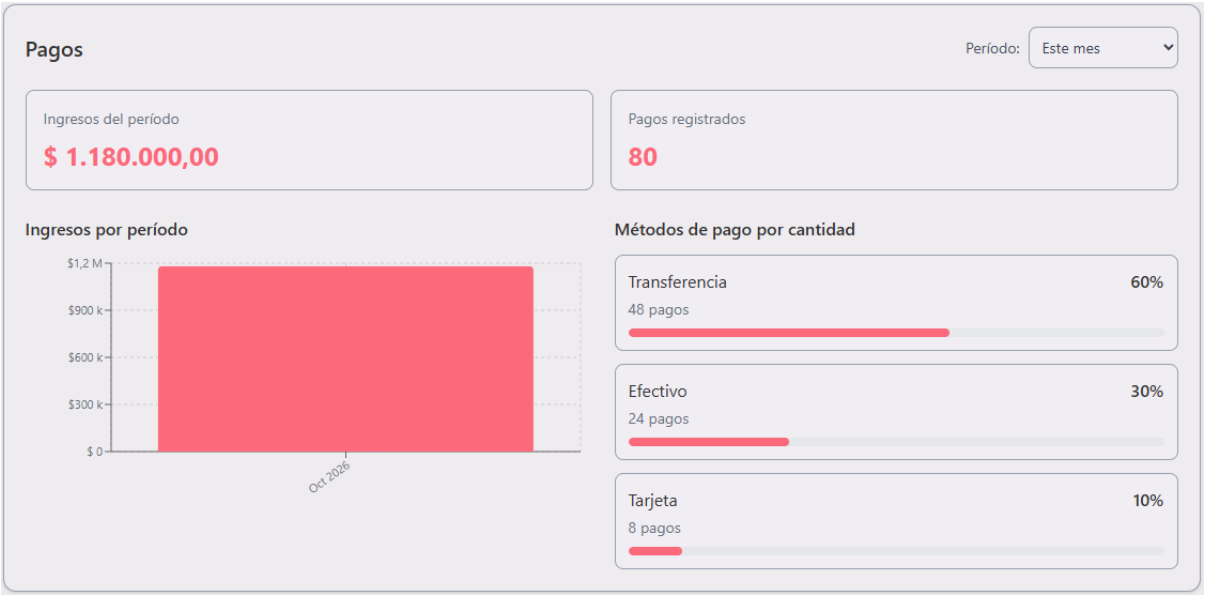
13.4. Estadísticas de profesores

Profesores totales activos. Clases dictadas según el período. Actividades totales. Clases dictadas por profesor y actividades.



13.4. Estadísticas de pagos

Ingresos registrados por período y los pagos realizados. Gráfico de ingresos por período en pesos argentinos. Cantidad y porcentaje por medio de pago. Los datos considerados corresponden a pagos de alumnos al negocio con tipo STUDENT y estado Paid. No representan pagos de suscripción que el negocio realiza a TurnoFácil.



14. Manuales e instructivos

14.1. Requisitos previos

Contar con una cuenta y haber iniciado sesión. Pertenecer como alumno al negocio correspondiente. Tener un plan y ciclo válidos, cuota al día y clases disponibles en el ciclo. Elegir una clase futura con cupo disponible y sin una reserva previa para esa misma clase.

14.2. Cómo reservar una clase

Ingresar al negocio correspondiente. Abrir la sección «Clases». Consultar las clases disponibles y seleccionar la fecha deseada. Revisar en la tarjeta o detalle la actividad,

el profesor, la fecha, el horario y los cupos. Seleccionar la clase y pulsar «Entrar a la clase». Si la operación es válida, esperar el mensaje «¡Te uniste a la clase correctamente!». Abrir «Reservas» o «Mis reservas» para comprobar que la reserva figure en el listado.

14.3. Si la clase está completa

Cuando no quedan cupos, la interfaz puede ofrecer «Entrar en lista de espera». La lista de espera no garantiza una reserva. Si se libera un lugar, el sistema puede enviar un correo a los alumnos anotados; la asignación no se realiza automáticamente.

14.4. Si la cuota está vencida o el plan no es válido

El sistema puede rechazar la reserva si la cuota está vencida, no existe un plan activo, no hay un ciclo activo o se alcanzó el límite de clases del ciclo. El usuario debe volver al espacio del negocio y consultar la sección «Pagos» o comunicarse con la administración para verificar su situación.

14.5. Cómo cancelar una reserva

Desde las reservas, seleccionar la reserva propia y utilizar la opción de cancelar si está disponible. El sistema no permite cancelar durante los 20 minutos previos al inicio de la clase. Si la cancelación se acepta, el registro se elimina.

15. Anexos

15.1. Arquitectura

TurnoFácil utiliza una arquitectura cliente-servidor web con separación por capas. El frontend implementado con React presenta las pantallas, captura acciones del usuario y consulta la API. Axios realiza las solicitudes HTTP y TanStack Query administra consultas y mutaciones remotas. El backend ASP.NET Core expone controladores REST y delega las reglas de negocio en servicios. Los repositorios y ApplicationDbContext permiten acceder a la base de datos PostgreSQL mediante Entity Framework Core.

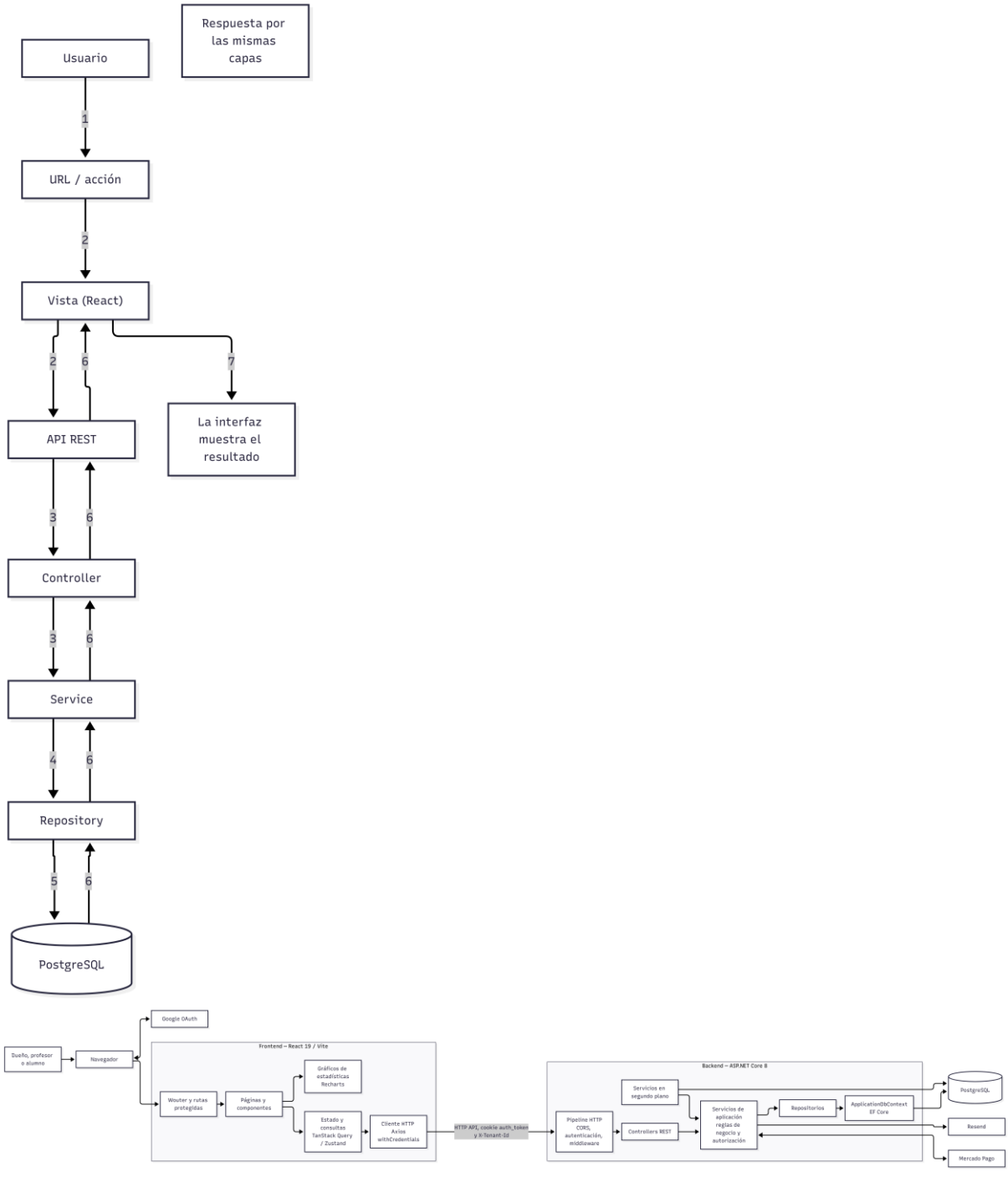
Flujo general de una solicitud:

1. El usuario realiza una acción en la interfaz React.
2. Axios envía una solicitud a la API; en los flujos multi-tenant se envía el identificador del negocio mediante X-Tenant-Id.
3. La autenticación y los middlewares validan la solicitud y establecen o comprueban el contexto correspondiente.
4. El controlador recibe la solicitud y delega la operación al servicio.
5. El servicio verifica permisos y reglas de negocio, y utiliza la capa de persistencia.
6. Entity Framework Core consulta o modifica PostgreSQL.
7. La API devuelve el resultado o un error y el frontend actualiza la interfaz.

Componentes relevantes: Program.cs configura los servicios y middlewares; TenantMiddleware.cs valida contexto de tenant y acceso en los flujos correspondientes; CurrentTenantService expone el contexto del negocio; ExceptionHandlingMiddleware.cs procesa excepciones centralizadas; Mapping.cs contiene perfiles de mapeo; ApplicationDbContext.cs define el acceso a los datos.

El backend se configura para ejecutarse en Docker sobre .NET 8 y exponer el puerto 8080. La configuración de producción del frontend debe documentarse según el hosting realmente utilizado.

Importante: no se identificó un filtro global de Entity Framework Core que aplique automáticamente el tenant a todas las entidades. El aislamiento depende de las comprobaciones del middleware y de los filtros y validaciones en servicios; por ello, los endpoints y servicios deben verificar correctamente el contexto de tenant.



15.1.2. Hosting del frontend y esquema de producción

TurnoFácil se despliega en un servidor virtual privado (VPS) contratado en Hetzner, con Ubuntu 24.04 como sistema operativo. La infraestructura permite centralizar el

alojamiento del frontend y del backend, manteniendo separadas sus responsabilidades.

El frontend está desarrollado con React y Vite. Durante la compilación, Vite genera archivos estáticos HTML, CSS y JavaScript optimizados para producción. Estos archivos son servidos por Nginx, que actúa como servidor web y punto de entrada a la aplicación.

El dominio turnofacilapp.com.ar utiliza Cloudflare para la administración de DNS. Cuando el proxy de Cloudflare está habilitado para el dominio, las solicitudes pasan por su infraestructura antes de llegar al VPS. Esto permite aplicar las funciones de protección y gestión del tráfico que ofrece el servicio.

Nginx recibe las solicitudes que llegan al servidor y las distribuye según su destino:

- Las solicitudes correspondientes a la interfaz web reciben los archivos estáticos del frontend.
- Las solicitudes dirigidas a /api/ se reenvían al backend ASP.NET Core mediante un proxy inverso.

El backend se encuentra preparado para ejecutarse en un contenedor Docker sobre .NET 8, utilizando el puerto 8080 para recibir solicitudes. En la configuración de producción, este puerto debe estar disponible para Nginx sin necesidad de exponerlo directamente a Internet.

La API utiliza Entity Framework Core para acceder a PostgreSQL mediante ApplicationDbContext. La base de datos almacena la información persistente del sistema y debe contar con restricciones de acceso que impidan conexiones desde orígenes no autorizados.

El flujo de una solicitud en producción es el siguiente:

1. El usuario accede a TurnoFácil desde su navegador mediante HTTPS.
2. Cloudflare resuelve el dominio y, cuando su proxy está habilitado, recibe y reenvía la solicitud al VPS de Hetzner.
3. Nginx recibe la solicitud y determina si debe servir un recurso del frontend o reenviarla a la API.
4. Si la solicitud corresponde a la API, Nginx la reenvía al backend ASP.NET Core.

5. El backend procesa la operación, aplica las validaciones y los permisos correspondientes y, cuando es necesario, consulta o modifica PostgreSQL.
6. La respuesta regresa a través de Nginx y Cloudflare hasta llegar al navegador.

En producción, Axios utiliza /api como dirección base de la API. Al utilizar rutas relativas, el frontend puede comunicarse con el backend a través del mismo dominio, simplificando la configuración de las solicitudes y el uso de cookies de autenticación.

La seguridad de las comunicaciones depende de la configuración TLS entre el navegador, Cloudflare y el servidor de origen. HTTPS protege los datos transmitidos durante cada conexión cifrada, mientras que la autenticación, la autorización y las validaciones del backend controlan el acceso a las operaciones y a la información del sistema.

15.2. Seguridad

15.2.1. Autenticación

La aplicación dispone de autenticación basada en JWT. En los flujos que lo utilizan, el token puede transportarse en una cookie llamada auth_token con atributos de seguridad configurados por el backend. Axios está configurado para enviar credenciales cuando corresponde. También existe autenticación con Google y flujos de verificación de correo y recuperación de contraseña.

15.2.2. Autorización y permisos

La autorización combina la identidad autenticada, el rol, el contexto de tenant y los permisos de la operación. Los permisos de grupos se administran mediante las entidades de grupos y permisos correspondientes; los profesores también pueden tener permisos individuales. El comportamiento efectivo debe documentarse de acuerdo con los permisos y reglas que estén habilitados en la versión entregada.

15.2.3. Aislamiento multi-tenant

El frontend identifica el negocio seleccionado mediante la ruta y envía X-Tenant-Id en las solicitudes que lo requieren. Ese encabezado identifica el contexto solicitado, pero no es una credencial ni concede autorización. El backend debe validar que el usuario pueda acceder al tenant, que el recurso pertenezca al tenant y que se cumplan los

permisos requeridos. El estado inactivo de un negocio o profesor también debe impedir el acceso en los flujos donde esa regla está implementada.

15.2.4. Protección de credenciales y datos

- Las contraseñas se procesan mediante BCrypt.Net-Next.
- Los tokens JWT se validan en el backend.
- Los tokens de Mercado Pago se protegen antes de almacenarse, según la implementación revisada.
- Las credenciales de proveedores externos deben configurarse fuera del código fuente y mantenerse privadas.
- Los errores inesperados se registran y se devuelve un mensaje genérico mediante el middleware de manejo de excepciones, aunque algunos controladores gestionan errores localmente.

15.2.5. Gestión de usuarios, grupos y recuperación

La aplicación incluye operaciones de cuenta, gestión de usuarios, grupos y permisos según las funciones implementadas, además de cambio y recuperación de contraseña. Los permisos efectivos de cada usuario dependen del rol, la pertenencia al tenant y las asignaciones vigentes.

La guía académica menciona la administración de usuarios, grupos y permisos. En el documento final deben adjuntarse capturas de las pantallas reales que existan en la versión entregada y no describir operaciones que no estén disponibles.